Structure d'un programme - modules

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Introduction aux modules **3**

A. Introduction aux modules 3

B. Exercice : Quiz 4

III. Création et utilisation de modules personnalisés **7**

A. Création et utilisation de modules personnalisés 7

B. Exercice : Quiz 11

IV. Utilisation des modules tiers **13**

A. Utilisation des modules tiers 13

B. Exercice : Quiz 16

V. Programmation modulaire avancée **18**

A. Programmation modulaire avancée 18

B. Exercice : Quiz 20

VI. Essentiel **22**

VII. Auto-évaluation **22**

A. Exercice 22

B. Test 24

Solutions des exercices **26**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Contexte

Dans le monde de la programmation, il est rare qu'un projet soit développé entièrement à partir de zéro sans réutilisation de code. L'un des grands avantages de Python réside dans sa capacité à organiser le code en modules. Un module est un fichier contenant des définitions et des instructions en Python (comme des fonctions, des classes, ou des variables) qui peuvent être importées et réutilisées dans d'autres fichiers. Cette organisation rend le développement plus modulaire, facilitant la gestion des projets complexes et la réutilisation du code.

Imaginons que vous préparez une recette de cuisine. Au lieu de créer chaque ingrédient de zéro, vous utilisez des ingrédients préemballés disponibles dans le commerce (comme de la farine, du sucre, des épices, etc.). Ces ingrédients préemballés sont comme des modules en Python : ils sont déjà prêts et vous pouvez les utiliser à tout moment pour créer votre plat. Vous n'avez pas besoin de les recréer, il vous suffit de les importer dans votre recette.

De la même manière, en Python, les modules permettent de structurer et d'organiser le code, mais surtout de réutiliser des fonctionnalités existantes. Vous pouvez soit créer vos propres modules pour structurer un projet, soit utiliser des modules préconstruits offerts par Python (ou des bibliothèques tierces) pour enrichir votre application.

II. Introduction aux modules

A. Introduction aux modules

Définition **Qu'est-ce qu'un module ? À quoi cela peut servir ?**

Imaginez que nous construisons une maison. Chaque pièce a une fonction spécifique : la cuisine pour cuisiner, le salon pour se détendre, la chambre pour dormir.

En Python, les modules fonctionnent un peu comme ces pièces. Ils nous permettent d'organiser notre code en blocs logiques, chacun ayant sa propre responsabilité. Ils peuvent contenir des fonctions, classes ainsi que des variables et sont rangé dans un package qui serait comme une « *catégorie* » de la pièce.

Un package peut comporter plusieurs module, il contient également un fichier __init__.py qui indique à Python que le dossier est un package. Les packages sont très utile pour structurer votre code.

Les cas d'utilisations ou un module est utile

L'utilisation de modules est non seulement une bonne pratique, mais elle est aussi recommandée pour plusieurs raisons :

- **Réutiliser** : au lieu de copier-coller le même morceau de code dans plusieurs fichiers, nous pouvons simplement écrire ce code une fois dans un module, puis l'importer où que nous en avons besoin. Cela réduit non seulement le risque d'erreurs, mais rend aussi le code plus facile à maintenir. Si nous avons besoins de modifier le comportement d'une fonction, nous n'avons qu'à le faire une seule fois dans le module, plutôt que dans chaque fichier où cette fonction est utilisée.
- **Organiser** : en divisant votre code en modules distincts, chacun dédié à une tâche spécifique, nous rendons le projet plus lisible et plus facile à comprendre. Par exemple, nous pourrions avoir un module pour gérer les calculs mathématiques (calcul.py), un autre pour traiter les entrées et sorties (io.py), et ainsi de suite.
- **Partager** : supposons que nous avons écrit un module pour analyser les fichiers CSV. Il est possible de facilement réutiliser ce même module dans plusieurs projets sans avoir à réécrire ou adapter le code.

Pour comprendre comment cela fonctionne, commençons par créer un module simple. Prenons un fichier Python, disons que nous l'appelons « *calcul.py* ». Dans ce fichier, on peut écrire une fonction simple qui additionne deux nombres :

```
1 # fichier : calcul.py
2 def addition(a, b):
3     return a + b
```

Voilà, nous venons de créer notre premier module ! Oui, c'est aussi simple que cela. Maintenant, nous pouvons l'utiliser dans un autre script Python en l'important. Disons que nous avons un autre fichier, « *main.py* », dans lequel nous souhaitons utiliser la fonction « *addition* » que nous effectué ensemble :

```
1 # fichier : main.py
2 import calcul
3 resultat = calcul.addition(5, 7)
4 print(f"Le résultat de l'addition est : {resultat}")
```

Lorsqu'on exécute `main.py`, il affiche : "Le résultat de l'addition est : 12". La magie opère grâce à l'instruction `import calcul`, qui dit à Python de charger tout le code du fichier `calcul.py` et de le rendre disponible pour utilisation.

Python ne se limite pas aux modules que nous écrivons. Le langage propose une vaste bibliothèque de modules préinstallés, que l'on appelle la « *bibliothèque standard* ». Il n'y a pas besoin de les installer ou de les télécharger ; ils sont déjà là, prêts à être utilisés. Par exemple, supposons que nous voulons calculer la racine carrée d'un nombre. Python a un module « *math* » qui fait cela et bien plus encore.

```
1 import math
2
3 nombre = 16
4 racine_carre = math.sqrt(nombre)
5 print(f"La racine carrée de {nombre} est : {racine_carre}")
```

Quand on exécute ce code, il affiche : "Le résultat de l'addition est : 12". La magie opère grâce à l'instruction `import calcul`, qui dit à Python de charger tout le code du fichier `calcul.py` et de le rendre disponible pour utilisation.

En plus de ces modules de la bibliothèque standard, il existe des milliers de modules créés par la communauté Python, disponibles via le gestionnaire de packages `*pip*`. Prenons l'exemple du module `*requests*`, très populaire pour effectuer des requêtes HTTP, c'est-à-dire pour communiquer avec des sites web et récupérer des données. Pour l'installer, il suffit d'ouvrir votre terminal ou votre invite de commande et de taper :

```
1 pip install requests
```

Et voilà ! Nous avons installé le module `*requests*`. Maintenant, voyons comment l'utiliser pour récupérer le contenu d'une page web :

```
1 import requests
2
3 reponse = requests.get("https://api.github.com")
4 print(f"Statut de la réponse : {reponse.status_code}")
5 print(f"Contenu de la réponse : {reponse.text}")
```

Ce code envoie une requête à l'API de GitHub et imprime le statut et le contenu de la réponse.

B. Exercice : Quiz

[solution n°1 p.27]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Qu'est-ce qu'un module en Python ?

- Une fonction prédéfinie du langage Python
- Un fichier contenant du code Python (fonctions, classes, variables) pouvant être importé et utilisé dans d'autres scripts
- Une méthode pour optimiser la performance de Python
- Un type de boucle spécifique à Python

Question 2

Pourquoi est-il recommandé d'utiliser des modules en Python ?

- Pour éviter d'écrire du code
- Pour simplifier l'installation de Python
- Pour organiser le code, favoriser la réutilisation et éviter les conflits de noms
- Pour améliorer la vitesse de compilation

Question 3

Quel mot-clé est utilisé pour importer un module en Python ?

- include
- using
- import
- require

Question 4

Que se passe-t-il lorsque vous exécutez `import math` dans un script Python ?

- Vous créez un nouveau module appelé `math`
- Vous téléchargez le module `math` depuis Internet
- Vous importez le module `math` de la bibliothèque standard de Python
- Vous supprimez toutes les variables mathématiques

Question 5

Comment appelle-t-on un répertoire contenant plusieurs modules Python ainsi qu'un fichier `__init__.py` ?

- Un projet
- Un module principal
- Un package
- Une fonction

Question 6

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la fonction principale du fichier __init__.py dans un package Python ?

- Empêcher le package d'être importé
- Permettre à Python de traiter le répertoire comme un package
- Installer automatiquement tous les modules du package
- Optimiser les performances du package

Question 7

Quelle commande utilisez-vous pour installer un module tiers en Python ?

- python install
- pip install
- module install
- module install

Question 8

Quel est le résultat de l'exécution du code suivant ?

```
1 import calcul
2 print(calcul.addition(5, 7))
```

Considérez que le module `calcul` contient une fonction `addition(a, b)` qui retourne la somme de `a` et `b`.

- Le programme ne s'exécutera pas car le module `calcul` n'est pas défini
- Le programme affiche : `57`
- Le programme affiche : `12`
- Le programme génère une erreur de syntaxe

Question 9

Pourquoi est-il utile d'utiliser le module `requests` en Python ?

- Pour manipuler les fichiers locaux sur l'ordinateur
- Pour effectuer des requêtes HTTP, comme récupérer des données d'un site web
- Pour gérer les expressions régulières dans les chaînes de caractères
- Pour améliorer la performance de l'application Python

Question 10

Quelle est la différence principale entre l'utilisation de import module et from module import * ?

- import module importe le module entier, tandis que from module import * importe uniquement le nom du module
- import module importe le module entier, tandis que from module import * importe toutes les fonctions et classes du module dans l'espace de noms actuel
- import module est obsolète et ne devrait plus être utilisé
- Les deux sont exactement identiques et peuvent être utilisés de manière interchangeable

Tous droits réservés à STUDI - Reproduction interdite

III. Création et utilisation de modules personnalisés

A. Création et utilisation de modules personnalisés

Définition **Comment créer des modules**

Les modules Python sont importants pour structurer et organiser le code dans un projet de manière modulaire et réutilisable. En séparant les différentes fonctionnalités dans des fichiers distincts, il est possible de rendre un code plus maintenable et plus facile à comprendre. L'utilisation de modules devient particulièrement cruciale dans les projets plus vastes.

Un module est un fichier Python qui contient des fonctions, des classes ou des variables. Il peut ensuite être importé dans d'autres fichiers pour être utilisé. Cette organisation réduit la duplication de code et favorise la réutilisation.

Méthode **Illustrons la création du module**

Commençons par créer un module qui contient des fonctions mathématiques basiques. Ce module sera stocké dans un fichier appelé `math_operations.py`.

```
1 # fichier : math_operations.py
2
3 def addition(a, b):
4     return a + b
5
6 def soustraction(a, b):
7     return a - b
8
9 def multiplication(a, b):
10     return a * b
11
12 def division(a, b):
13     if b == 0:
14         return "Erreur : division par zéro"
15     return a / b
```

Ce module contient quatre fonctions qui réalisent des opérations mathématiques simples : addition, soustraction, multiplication et division.

Pour utiliser ce module dans un autre fichier, il suffit de l'importer dans le script principal avec l'instruction `import`. Cela permet d'accéder à toutes les fonctions définies dans le module.

```
1 # fichier : main.py
2 import math_operations
3
4 resultat = math_operations.addition(10, 5)
5 print(f"Résultat de l'addition : {resultat}")  # Affiche : Résultat de l'addition : 15
```

Nous importons le module entier en utilisant `*import math_operations*`, puis nous appelons les fonctions avec le nom du module comme préfixe.

Remarque

Si nous avons besoin que de certaines fonctions d'un module, il est possible d'importer que les fonctions spécifiques avec l'instruction `*from ... import*`. Cela simplifie le code en permettant d'utiliser directement le nom de la fonction sans le préfixe du module.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Les subtilités de l'importation des modules**

```
1 from math_operations import addition, division
2
3 resultat = addition(8, 2)
4 print(f"Résultat de l'addition : {resultat}")  # Affiche : Résultat de l'addition : 10
```

Il est possible également d'importer toutes les fonctions d'un module : Cela se fait avec `from ... import *`. Cependant, il est important de noter que cette méthode peut entraîner des conflits de noms si plusieurs modules contiennent des fonctions ayant le même nom.

```
1 from math_operations import *
2 resultat = multiplication(6, 7)
3 print(f"Résultat de la multiplication : {resultat}")  # Affiche : Résultat de la
  multiplication : 42
```

Pour schématiser le cas des soucis d'imports, admettons que nous avons la fonction *addition* dans deux modules python : *math_operations.py* et *string_operations.py*

```
1 # fichier : math_operations.py
2 def addition(a, b):
3     return a + b
4
5 # fichier : string_operations.py
6 def addition(a, b):
7     return a + " " + b
```

Le souci avec le code suivant, c'est que Python ne sait pas faire la différence entre les deux méthodes, si on importe les deux. On appelle cela un « *conflit de nom* ».

```
1 from math_operations import *
2 from string_operations import *
3
4 resultat = addition(10, 5)
5 print(resultat)
```

La solution recommandée par Python est de ne pas utiliser le « *import* * » , mais de privilégier d'importer la méthode. Si on désire importer tout de même les deux méthodes addition, nous pouvons les renommer lors de l'imports.

```
1 from math_operations import addition as addition_numerique
2 from string_operations import addition as addition_chaine
3
4 resultat_math = addition_numerique(10, 5)
5 resultat_string = addition_chaine("Hello", "World")
```

Fondamental

Lorsque nous travaillons sur des projets plus grands, l'organisation des modules devient essentielle. Les packages vont permettre de regrouper plusieurs modules dans des dossiers pour structurer le projet de manière plus logique et modulaire.

Définition **Structure d'un projet python avec des modules et package**

Un **package** en Python est un répertoire qui contient plusieurs modules Python ainsi qu'un fichier spécial appelé __init__.py. Ce fichier indique à Python que le répertoire doit être traité comme un package, c'est-à-dire une unité de regroupement de plusieurs modules. Un package peut lui-même contenir des sous-packages pour organiser encore plus le projet.

Tous droits réservés à STUDI - Reproduction interdite

```
mon_projet/
│
├── main.py            # Script principal
├── math_operations.py # Module pour les opérations mathématiques
└── utils/             # Package pour les utilitaires
    ├── __init__.py    # Fichier d'initialisation du package utils
    └── io_utils.py    # Module pour la gestion des fichiers
```

Figure 1 : structure de projet avec des packages

Dans cet exemple, nous avons :

- **main.py** : le fichier principal où le programme s'exécute.
- **math_operations.py** : un module qui contient des fonctions pour effectuer des calculs mathématiques.
- **utils/** : un package contenant des modules utilitaires comme io_utils.py pour la gestion des fichiers.

Le fichier __init__.py peut être vide, mais son existence indique à Python que le répertoire doit être considéré comme un package. Ce fichier est également un endroit idéal pour inclure du code d'initialisation du package ou pour pré-importer certains éléments du package afin de les rendre plus facilement accessibles.

Méthode

Par exemple, nous pouvons utiliser __init__.py pour exposer certaines fonctions ou classes depuis les modules internes du package, facilitant ainsi leur importation à partir du package principal :

```
# fichier : utils/__init__.py
from .io_utils import lire_fichier
```

Maintenant, nous pouvons importer directement la fonction lire_fichier depuis le package utils, au lieu de devoir passer par le module *io_utils* :

```
from utils import lire_fichier
```

Il existe deux manières principales d'importer des modules dans un projet : les **importations absolues** et les **importations relatives** :

Définition

Une **importation absolue** spécifie le chemin complet du module ou du package, depuis la racine du projet. Cela rend le code clair et explicite, car il est facile de voir d'où provient chaque fonction ou classe importée. C'est la méthode recommandée pour les grands projets car elle réduit le risque de confusion.

```
# fichier : main.py
from utils.io_utils import lire_fichier
```

Cette méthode est adaptée aux projets avec plusieurs sous-modules ou sous-packages. On peut savoir exactement où chaque fonction se trouve dans la structure du projet.

L'importation relative est une autre manière d'importer des modules, basée sur la position relative du module que nous voulons importer par rapport au fichier dans lequel nous nous trouvons. Les points *(.)* sont utilisés pour indiquer le niveau de profondeur par rapport à la hiérarchie des dossiers.

Tous droits réservés à STUDI - Reproduction interdite

```
1 # fichier : io_utils.py
2 from . import lire_fichier
```

Cette méthode est utile pour importer des éléments au sein d'un même package ou pour réduire la longueur des imports. Cependant, les importations relatives peuvent devenir confuses dans les projets complexes avec plusieurs niveaux de sous-packages. Il est recommandé de privilégier les importations absolues pour plus de clarté.

Remarque

A contrario, un souci d'import en Python assez classique, est l'import cyclique. Il s'agit de deux modules qui s'importent mutuellement. Cela peut entraîner des erreurs et des comportements inattendus. Si on doit casser une dépendance circulaire, on peut utiliser des importations différées, c'est-à-dire importer le module au moment où on en a réellement besoin dans une fonction :

```
1 # Importation différée pour éviter les cycles
2 def utiliser_fonction_de_b():
3 from module_b import fonction
4 fonction()
```

Pour éviter ce souci, l'importation absolue reste à privilégier.

Méthode **Bonnes pratiques en Python concernant les modules et package**

En python, il est recommandé de suivre ces bonnes pratiques :

- **Organisez le projet en modules et packages** : gardez votre code modulaire en le répartissant dans des fichiers logiques. Cela rendra votre code plus maintenable et évolutif. Utilisez des packages pour regrouper des modules par fonctionnalité.
- **N'utilisez pas `*from ... import **` dans les grands projets** : bien que cela puisse sembler pratique, cela surcharge l'espace de noms et peut causer des conflits de noms. Il est toujours préférable d'importer les fonctions de manière explicite.
- **Évitez les imports cycliques** : assurez-vous que vos modules ne s'importent pas mutuellement pour éviter des erreurs difficiles à déboguer. Si nécessaire, utilisez des importations différées.
- **Utilisez des imports absolus** : privilégiez les imports absolus plutôt que relatifs pour garder votre code clair et lisible. Les imports absolus sont moins sujets à confusion, surtout dans les projets complexes.

Pour concrétiser ces concepts, imaginons que José, gestionnaire d'un magasin, souhaite développer une application pour suivre les employés, calculer leur salaire en fonction des heures travaillées, et générer un rapport hebdomadaire. Nous allons l'aider à structurer son projet en plusieurs modules.

José commence par créer un module pour gérer les informations de ses employés. Ce module contient des fonctions pour ajouter des employés et afficher leurs informations.

```
1 # fichier : gestion_employes.py
2 # gestion_employes.py
3
4 employes = []
5
6 def ajouter_employe(nom, taux_horaire, heures_travaillees):
7     employe = {"nom": nom, "taux_horaire": taux_horaire, "heure_travaillees":
heures_travaillees}
8     employes.append(employe)
9
10 def afficher_employes():
11     return employes
```

Ensuite, il crée un module pour calculer les salaires des employés en fonction des heures travaillées.

Tous droits réservés à STUDI - Reproduction interdite

```python
# calcul_salaire.py

def calculer_salaire(employe):
    return employe["taux_horaire"] * employe["heure_travaillees"]
```

Enfin, José crée un module qui génère un rapport hebdomadaire récapitulant les performances des employés et leurs salaires.

```python
# rapport.py
from calcul_salaire import calculer_salaire

def generer_rapport(employes):
    for employe in employes:
        salaire = calculer_salaire(employe)
        print(f"{employe['nom']} - Heures: {employe['heure_travaillees']}, Salaire: {salaire} euros")
```

José intègre tous les modules dans son script principal « *main.py* » :

```python
# main.py
from gestion_employes import ajouter_employe, afficher_employes
from rapport import generer_rapport

if __name__ == "__main__":
    # Ajout des employés
    ajouter_employe("José", 20, 40)
    ajouter_employe("Marie", 25, 35)

    # Récupérer la liste des employés
    employes = afficher_employes()

    # Générer le rapport
    generer_rapport(employes)
```

Lorsque José exécute ce script, il obtient un rapport détaillant les informations des employés, leurs heures travaillées et leurs salaires :

```
Employé Alice ajouté avec un taux horaire de 15 euros.
Employé Bob ajouté avec un taux horaire de 12 euros.
Nom : Alice, Taux Horaire : 15, Heures Travaillées : 40
Nom : Bob, Taux Horaire : 12, Heures Travaillées : 35
Rapport Hebdomadaire :
Alice - Heures : 40 - Salaire : 600 euros
Bob - Heures : 35 - Salaire : 420 euros
```

B. Exercice : Quiz

[solution n°2 p.29]

Question 1

Quel est le rôle principal du fichier __init__.py dans un package Python ?

- ○ Il doit contenir toutes les fonctions du package
- ○ Il empêche le package d'être importé
- ○ Il indique à Python que le répertoire est un package
- ○ Il permet de configurer les variables globales du projet

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Que permet l'instruction from math_operations import * ?

- Elle importe seulement les fonctions spécifiées de math_operations
- Elle importe toutes les fonctions et classes de math_operations sans utiliser le préfixe math_operations
- Elle importe uniquement les variables globales définies dans math_operations
- Elle crée une copie du module math_operations

Question 3

Quelle méthode est recommandée pour importer des modules dans des projets complexes ?

- Utiliser from module import * pour chaque module
- Utiliser import module_name ou importer des fonctions spécifiques avec from module import fonction
- Tout inclure dans un seul fichier pour éviter les imports
- Importer des modules à la fin du script

Question 4

Pourquoi est-il déconseillé d'utiliser from ... import * dans un projet avec plusieurs modules ?

- Cela réduit les performances du projet
- Cela peut entraîner des conflits de noms si plusieurs modules ont des fonctions identiques
- Cela rend impossible l'importation de classes
- Cela empêche Python de trouver les modules dans la bibliothèque standard

Question 5

Quelle est la différence entre un module et un package en Python ?

- Un module est une collection de packages
- Un package est un répertoire contenant plusieurs modules et un fichier __init__.py
- Un package est un fichier Python unique
- Un module ne peut contenir que des classes, alors qu'un package contient des fonctions

Question 6

Comment appelle-t-on l'importation basée sur la position relative des modules ?

- Importation différée
- Importation absolue
- Importation relative
- Importation cyclique

Question 7

Tous droits réservés à STUDI - Reproduction interdite

Comment créer un module Python ?

- En écrivant du code dans un fichier avec l'extension .py
- En créant un répertoire contenant un fichier __init__.py
- En exécutant pip install module_name
- En utilisant la commande create module dans Python

Question 8

Quelle est la principale raison de structurer un projet en modules et packages ?

- Accélérer le développement
- Réduire la taille du fichier .pyc
- Organiser le code pour le rendre plus maintenable, réutilisable et lisible
- Obtenir un meilleur support de la part de la communauté Python

Question 9

Que permet l'instruction import module_name ?

- Elle exécute le fichier spécifié dans le module
- Elle charge le module dans le script actuel, permettant d'utiliser ses fonctions et classes
- Elle renomme le module
- Elle importe uniquement les classes du module

Question 10

Qu'est-ce qu'un conflit de noms en Python ?

- Lorsque deux modules portent le même nom
- Lorsque deux fonctions ou variables avec le même nom existent dans le même espace de noms
- Lorsque un fichier Python a le même nom qu'un fichier de configuration
- Lorsque deux packages sont importés en même temps

IV. Utilisation des modules tiers

A. Utilisation des modules tiers

Définition **Qu'est-ce qu'un module tiers ?**

Il est possible également d'utiliser des modules développés par d'autre personne : ils sont disponibles sur le Python Package Index (PyPI) et couvrent une grande variété de besoins : requêtes HTTP, manipulation de fichiers, traitement de données, interaction avec des bases de données, etc.

`pip` est l'outil standard pour gérer l'installation de packages tiers. Pour installer un module depuis PyPI, il suffit d'utiliser la commande suivante :

```
pip install nom_du_package
```

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Prenons un exemple concret avec le package « *requests* », utilisé pour faire des requêtes HTTP. Pour l'installer, exécutez :

```
pip install requests
```

Une fois le package installé, nous pouvons l'importer dans notre code et l'utiliser. Voici un exemple pour envoyer une requête GET et afficher le contenu JSON d'une API :

```
import requests

response = requests.get('https://jsonplaceholder.typicode.com/posts/1')

if response.status_code == 200:
    print(response.json())  # Affiche le contenu JSON
else:
    print(f"Erreur {response.status_code}")
```

Méthode

Voyons quelques modules qui sont énormément utilisés en Python :

Le module os

Le module **os** fournit une interface pour interagir avec le système d'exploitation. Il permet de manipuler les fichiers, les répertoires, et d'exécuter des commandes système.

```
import os

# Afficher le répertoire de travail actuel
print(os.getcwd())  # Renvoie le chemin du répertoire de travail actuel

# Lister les fichiers et dossiers dans un répertoire
print(os.listdir('.'))  # Liste les fichiers dans le répertoire courant
```

Le module random

Le module **random** permet de générer des nombres pseudo-aléatoires, de choisir des éléments au hasard dans une liste, et de simuler des scénarios comme des tirages au sort.

Voici comment nous pouvons l'utiliser pour générer des nombres aléatoires :

```
import random

# Générer un nombre flottant aléatoire entre 0 et 1
print(random.random())

# Générer un nombre entier aléatoire entre 1 et 100
print(random.randint(1, 100))

# Choisir un élément aléatoire dans une liste
noms = ['Alice', 'Bob', 'Charlie']
print(random.choice(noms))
```

Le module math

Le module **math** fournit des fonctions pour effectuer des calculs mathématiques complexes. Il est très utile dans les projets nécessitant des calculs scientifiques, financiers ou statistiques. Voyons quelques fonctions mathématique de base.

Tous droits réservés à STUDI - Reproduction interdite

```python
import math

# Calculer la racine carrée d'un nombre
print(math.sqrt(16))  # Renvoie 4.0

# Calculer le sinus d'un angle en radians
print(math.sin(math.pi / 2))  # Renvoie 1.0

# Trouver le plus grand entier inférieur ou égal à un nombre donné
print(math.floor(3.7))  # Renvoie 3
```

Maintenant, voyons la gestion des versions. Cette composante est essentiel car il peut arriver qu'une version d'un module est compatible avec noitre projet et non une autre : par defaut pip installe la dernière version du module.

La gestion des versions garantie la stabilité du projet : les modules tiers sont souvent mis à jour avec de nouvelles fonctionnalités ou des corrections de bugs, mais ces mises à jour peuvent parfois introduire des changements incompatibles. Il est donc important de figer les versions des modules utilisés dans vos projets.

Lorsqu'on installe un module, il est possible de spécifier la version que l'on souhaite utiliser. Cela nous permet de figer une version particulière, assurant ainsi que le projet ne sera pas affecté par une future mise à jour potentiellement incompatible.

Par exemple, pour installer la version 2.25.0 de `requests` :

```
pip install requests==2.25.0
```

Nous pouvons également définir une plage, dans l'exemple ci-dessous, nous voulons que pip installe une version de request supérieur ou égal à 2.25.0, mais dans tous les cas, inférieur à 3.0.

```
pip install requests>=2.25.0,<3.0
```

Cela garantit que seules les versions mineures (corrections et nouvelles fonctionnalités) seront prises en compte, et non les versions majeures, qui pourraient introduire des changements incompatibles.

Il est également possible de vérifier si des mises à jour sont disponibles pour les packages installés dans votre environnement avec la commande suivante :

```
pip list --outdated
```

Cela liste les modules dont la version installée est obsolète par rapport à la version disponible sur PyPI. Pour mettre à jour un module spécifique :

```
pip install --upgrade requests
```

Pour permettre aux autres développeurs d'avoir le même environnement que celui que le concepteur/conceptrice de l'application possède en locale, il est nécessaire de concevoir le fichier *'requirements.txt'*. Il permet de capturer les dépendances du projet et leurs versions spécifiques, garantissant que tous les membres de l'équipe et toutes les machines de déploiement utilisent les mêmes versions de packages.

Pour générer automatiquement un fichier `*requirements.txt*` basé sur les packages actuellement installés dans l'environnement, il suffit d'effectuer :

```
pip freeze > requirements.txt
```

Le fichier résultant liste tous les modules et leurs versions exactes :

```
requests==2.25.1
urllib3==1.26.5
```

Cela garantit que les versions des modules sont figées et que toute personne travaillant sur le projet utilisera les mêmes versions.

Pour installer toutes les dépendances listées dans un fichier `*requirements.txt*` sur une autre machine ou un autre environnement :

```
pip install -r requirements.txt
```

Cette commande installe les versions exactes spécifiées dans le fichier, assurant la cohérence de l'environnement.

Tous droits réservés à STUDI - Reproduction interdite

La gestion fait partie des bonnes pratiques, mais nous devons aussi :

1. **Éviter les modules inutilisés** : ne pas laisser des modules inutilisés dans le projet. Si on cesse d'utiliser un module, désinstallons-le avec `*pip uninstall*`.
2. **Vérifier régulièrement les mises à jour de sécurité** : certaines mises à jour corrigent des vulnérabilités de sécurité. Utilisez la commande `*pip list --outdated*` régulièrement et mettez à jour les packages critiques.
3. **Testez les nouvelles versions dans un environnement de test** : avant de mettre à jour un package dans un projet en production, testez-le dans un environnement de développement pour vérifier qu'il n'introduit pas de bugs ou d'incompatibilités.

Ce qui est déconseillé est d'installer les modules directement sur le système. Les environnements virtuels sont à privilégier.

B. Exercice : Quiz

[solution n°3 p.32]

Question 1

Quel module Python permet de gérer les fichiers et répertoires du système ?

- sys
- os
- file
- path

Question 2

Quelle commande permet d'installer un module tiers depuis le Python Package Index (PyPI) ?

- python install
- pip install
- py install
- pip freeze

Question 3

Que contient un fichier `requirements.txt` ?

- La documentation du projet
- Les fichiers exécutables du projet
- La liste des dépendances (modules) du projet avec leurs versions
- Les paramètres de configuration du projet

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la commande utilisée pour installer les modules spécifiés dans un fichier `requirements.txt` ?

- pip install requirements.txt
- pip download -r requirements.txt
- pip install -r requirements.txt
- python install -r requirements.txt

Question 5

Quel est l'avantage principal de figer les versions des modules dans un projet ?

- Cela permet d'accélérer l'installation des modules
- Cela permet de s'assurer que le projet est compatible avec les versions testées des modules
- Cela réduit la taille des fichiers du projet
- Cela désactive les mises à jour automatiques

Question 6

Que fait la commande `pip freeze` ?

- Elle supprime tous les modules installés dans l'environnement Python
- Elle met à jour tous les modules installés à leur dernière version
- Elle affiche la liste des modules installés avec leurs versions
- Elle bloque les modules pour empêcher leur mise à jour

Question 7

Quelle commande permet de voir quelles versions de packages sont obsolètes dans l'environnement Python ?

- pip list --outdated
- pip upgrade --list
- pip freeze --upgrade
- pip list --old

Question 8

Pourquoi est-il important d'utiliser un fichier requirements.txt dans un projet Python ?

- Pour réduire la taille du projet
- Pour améliorer la sécurité
- Pour garantir que tous les développeurs utilisent les mêmes versions de packages
- Pour accélérer les performances du projet

Question 9

Tous droits réservés à STUDI - Reproduction interdite

Pourquoi est-il recommandé d'utiliser un environnement virtuel pour installer des modules dans un projet Python ?

- Pour améliorer la vitesse d'exécution du code
- Pour isoler les dépendances d'un projet et éviter les conflits entre différents projets
- Pour faciliter l'installation des modules
- Pour réduire la taille des fichiers du projet

Question 10

Quelle est la différence entre pip install et pip install -r requirements.txt ?

- pip install installe un module, tandis que pip install -r requirements.txt installe plusieurs modules à partir du fichier requirements.txt
- pip install désinstalle un module, tandis que pip install -r requirements.txt installe un module
- pip install met à jour tous les modules, tandis que pip install -r requirements.txt les désinstalle
- pip install crée un fichier requirements.txt, tandis que pip install -r requirements.txt ne fait rien

V. Programmation modulaire avancée

A. Programmation modulaire avancée

Définition **La programmation modulaire**

La modularité est au cœur du développement Python, car elle permet d'organiser et de maintenir des projets complexes. Cette partie va aborder l'importation dynamique de modules en utilisant `*importlib*`, et l'importance de la sécurité dans la gestion des modules. Nous examinerons comment recharger un module pour refléter les modifications sans redémarrer l'interpréteur Python, puis nous explorerons les meilleures pratiques pour sécuriser l'importation des modules et éviter l'exécution de code non fiable.

Précédemment, nous avons visualisé comment importer des modules de manière manuel, mais, il est parfois nécessaire de charger des modules dynamiquement à l'exécution, surtout lorsqu'on ne connaît pas à l'avance les modules nécessaires. Le module `*importlib*` permet de charger et de recharger des modules à la volée, offrant une grande flexibilité.

Implémentation de la programmation modulaire

L'importation dynamique permet de charger des modules pendant l'exécution du programme, offrant ainsi une grande flexibilité dans des systèmes modulaires ou configurables. Au lieu d'importer un module au début du fichier, nous pouvons attendre d'avoir une condition ou un contexte spécifique pour déterminer quel module charger. Cette approche est couramment utilisée dans les projets configurables ou les applications avec des plugins.

Prenons un exemple où José souhaite concevoir un programme qui choisit le module à importer selon l'environnement de travail. Il utilise importlib.import_module pour accomplir cette tâche :

```
1 import importlib
2
3 def charger_module_dynamique(module_name):
4     try:
5         module = importlib.import_module(module_name)
6         return module
7     except ImportError:
8         print(f"Le module '{module_name}' n'a pas pu être importé.")
9
10 # Exemple d'utilisation
```


Tous droits réservés à STUDI - Reproduction interdite

```
11 module_name = 'math'
12 module = charger_module_dynamique(module_name)
13 if module:
14     print(module.sqrt(16))  # Affiche 4.0
```

Remarque

La capacité de importlib.import_module à accepter des chaînes de caractères comme noms de modules permet de créer des systèmes flexibles et modulaires. Toutefois, cette flexibilité doit être encadrée par des pratiques de sécurité strictes (que nous aborderons plus tard) pour éviter de charger des modules non approuvés ou malveillants.

Une subtilité importante à noter : l'importation dynamique retarde le chargement du module jusqu'au moment où il est réellement nécessaire, ce qui peut également aider à optimiser la performance des grands projets en retardant le chargement de modules inutiles.

Il est souvent nécessaire de modifier un module en cours de développement et de tester ces modifications sans redémarrer l'interpréteur Python. La fonction `*importlib.reload()*` permet de recharger un module après l'avoir modifié, sans quitter la session de développement.

Prenons un exemple où José travaille sur un module de configuration qu'il ajuste fréquemment. Pour éviter de redémarrer son environnement de développement après chaque modification, il peut utiliser `*reload()*` :

```
1 import importlib
2 import config  # Supposons que nous avons un module nommé "config"
3
4 # Modifier le fichier config.py, puis recharger
5 importlib.reload(config)
6
7 print(config.PARAMETRE)
```

Remarque

La subtilité est que `*reload()*` ne remplace que le contenu du module déjà importé, donc si des références à des objets de ce module ont déjà été prises ailleurs dans le programme, elles ne seront pas mises à jour. Cela peut entraîner des comportements inattendus.

Utilité de l'importation dynamique

L'importation dynamique est une technique puissante, mais elle présente aussi des risques si elle est mal utilisée. Le principal danger réside dans l'importation de modules non approuvés ou malveillants, qui pourraient contenir du code non sécurisé.

Pour remédier à cela, il est impératif de limiter les modules qui peuvent être chargés afin d'éviter l'exécution de code non approuvé. Cela est particulièrement vrai lorsque les utilisateurs ou des données externes influencent les modules à importer.

Dans l'exemple suivant, José met en place une liste blanche de modules autorisés à être importés dynamiquement pour éviter d'utiliser des modules non sécurisés.

```
1 import importlib
2
3 modules_autorises = ['math', 'random', 'os']
4
5 def charger_module_securise(module_name):
6     if module_name in modules_autorises:
7         return importlib.import_module(module_name)
8     else:
9         raise ValueError(f"Le module '{module_name}' n'est pas autorisé.")
```

Tous droits réservés à STUDI - Reproduction interdite

```
10
11 # Exemple sécurisé
12 module = charger_module_securise('math')
13 print(module.sqrt(25))  # Affiche 5.0
```

Remarque

Python exécute le code contenu dans un module au moment où celui-ci est importé. Cela signifie qu'un module contenant du code malveillant pourrait compromettre votre programme lors de l'importation. Il est donc impératif de ne jamais importer des modules provenant de sources non vérifiées.

Risques des modules téléchargés d'internet

Prenons un exemple : si José autorise des modules téléchargés d'Internet ou fournis par des utilisateurs à être importés directement, il court un risque. La meilleure approche est d'utiliser des environnements virtuels et d'importer des modules uniquement à partir de sources de confiance comme PyPI, en s'assurant que ceux-ci sont maintenus et validés.

Un environnement virtuel isole les dépendances d'un projet, garantissant que les modules tiers installés n'affectent pas l'environnement global du système. Cela réduit le risque d'interférence ou d'exécution de code malveillant.

Fondamental

L'importation dynamique avec `*importlib*` permet une grande flexibilité dans la gestion des modules Python, en offrant la possibilité de charger des modules en fonction du contexte, de les recharger sans redémarrer l'interpréteur, et de maintenir un système modulaire. Cependant, cette flexibilité doit être accompagnée de mesures de sécurité strictes, notamment en limitant les modules importables et en utilisant des environnements contrôlés.

B. Exercice : Quiz

[solution n°4 p.34]

Question 1

Quel est l'avantage principal de l'importation dynamique avec `importlib.import_module()` ?

- Elle permet de réduire la taille des fichiers du projet
- Elle permet de charger des modules uniquement au moment de leur utilisation
- Elle permet de désinstaller des modules à l'exécution
- Elle empêche l'importation de modules non sécurisés

Question 2

Quelle fonction du module `importlib` permet de recharger un module déjà importé ?

- importlib.import_module()
- importlib.load_module()
- importlib.reload()
- reload.module()

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Quel est le risque principal de l’importation dynamique non sécurisée de modules ?

- Provoquer des erreurs d'exécution
- Importer des modules inutiles
- Exécuter du code malveillant présent dans un module non approuvé
- Réduire les performances globales du programme

Question 4

Quel module Python est utilisé pour l'importation dynamique de modules ?

- os
- importlib
- math
- random

Question 5

Quel est le rôle principal de la fonction `importlib.import_module()` ?

- Recharger un module modifié
- Importer un module en utilisant une chaîne de caractères
- Supprimer un module non utilisé
- Lister les modules disponibles dans l'environnement

Question 6

Que se passe-t-il si un module est déjà importé et que vous essayez de le recharger avec `importlib.reload()` ?

- Le module est rechargé avec les nouvelles modifications
- Le module est désinstallé puis réinstallé
- Le module est rechargé mais ses objets existants ne sont pas mis à jour
- Le programme plante automatiquement

Question 7

Dans quel cas l’importation dynamique de modules est-elle particulièrement utile ?

- Lorsque le module doit être modifié après l'importation
- Lorsque le module à importer dépend d'une configuration ou d'un contexte spécifique
- Lorsque vous voulez accélérer l'importation d'un module
- Lorsque vous importez des modules depuis Internet

Question 8

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la meilleure pratique pour sécuriser l'importation dynamique de modules ?

- Importer tous les modules disponibles dans l'environnement
- Utiliser une liste blanche de modules autorisés
- Utiliser uniquement `import` au lieu de `importlib`
- Importer des modules directement depuis Internet

Question 9

Pourquoi est-il déconseillé d'importer des modules provenant de sources non fiables ?

- Cela peut ralentir l'exécution du programme
- Le code contenu dans ces modules peut être malveillant et compromette la sécurité
- Cela provoque des erreurs d'importation
- Cela surcharge la mémoire du programme

Question 10

Comment restreindre les modules qui peuvent être importés dynamiquement ?

- En limitant l'importation aux modules du système
- En définissant une liste blanche de modules approuvés
- En désactivant les imports dans le projet
- En évitant l'utilisation de `importlib`

VI. Essentiel

Les modules en Python sont comme des boîtes à outils bien rangées. Imaginons que nous rénovons une maison : nous avons une boîte pour les clous, une pour les vis, et une pour les outils. Chaque boîte contient des éléments spécifiques qui sont utilisés à des moments précis. De la même manière, les modules permettent de diviser le code en parties plus petites, chacune dédiée à un domaine particulier.

Par exemple, si on écrit une application pour un magasin en ligne, nous pouvons avoir un module pour gérer les produits, un autre pour les paiements et un troisième pour les clients. En important le module dont nous avons besoin, on évite de surcharger le code avec des fonctionnalités inutiles à ce moment-là, tout comme nous n'utilisons pas une perceuse pour serrer une vis.

Les modules Python peuvent être des modules intégrés, comme `math` pour les calculs ou `os` pour manipuler les fichiers système, ou des modules personnalisés que nous créons pour structurer le code. Il existe aussi des modules tiers que l'on peut installer avec `pip`, comme `requests` pour gérer les requêtes HTTP. Importer un module se fait simplement avec `import`, nous permettant d'accéder aux outils spécifiques que vous avez mis dans chaque boîte.

VII. Auto-évaluation

A. Exercice

José, qui gère un magasin en ligne, souhaite automatiser la gestion des commandes. Il vous demande de l'aider à structurer son projet en plusieurs modules pour rendre son application plus modulaire et maintenable. Vous allez créer plusieurs fichiers qui contiennent des fonctions spécifiques, utiliser des modules tiers et mettre en place un système de configuration dynamique.

Étape 1 : créer un module pour gérer les commandes

Tous droits réservés à STUDI - Reproduction interdite

José vous demande de créer un module appelé **commandes.py**, qui contient deux fonctions :

- **ajouter_commande(client, produit, prix)** : cette fonction doit prendre en paramètres le nom du client, le nom du produit, et son prix, puis enregistrer cette commande dans une liste globale appelée commandes.
- **afficher_commandes()** : cette fonction doit afficher toutes les commandes enregistrées dans la liste commandes.

Question 1

[solution n°5 p.37]

Créez le fichier **commandes.py** avec les deux fonctions demandées.

Indice :

La liste commandes doit être globale dans le module, et les fonctions doivent l'utiliser pour ajouter des éléments et les afficher.

Étape 2 : importer le module commandes

Maintenant que vous avez créé le module **commandes.py**, José veut que vous l'importiez dans un autre fichier appelé **main.py**, où vous utiliserez les fonctions pour ajouter quelques commandes.

Question 2

[solution n°6 p.37]

Dans le fichier **main.py**, importez le module commandes et ajoutez trois commandes avec les produits achetés par différents clients, puis affichez toutes les commandes.

Indice :

Vous pouvez utiliser import commandes pour importer le module, et appeler les fonctions avec commandes.ajouter_commande(...) et commandes.afficher_commandes().

Étape 3 : utiliser un module tiers pour la gestion des dates

José aimerait maintenant ajouter la date de chaque commande. Vous allez utiliser le module **datetime** (qui fait partie des modules standards) pour obtenir la date actuelle au moment de l'ajout d'une commande. Modifiez la fonction **ajouter_commande** dans le fichier **commandes.py** pour qu'elle ajoute la date de la commande automatiquement.

Question 3

[solution n°7 p.37]

Modifiez la fonction **ajouter_commande()** pour inclure la date de chaque commande en utilisant le module datetime.

Indice :

Utilisez datetime.datetime.now() pour obtenir la date et l'heure actuelles.

Étape 4 : utiliser un module tiers pour envoyer des requêtes

José souhaite également récupérer des informations sur ses produits à partir d'une API externe. Utilisez le module **requests** pour envoyer une requête GET à l'URL suivante : typicode[1], puis affichez le contenu de la réponse.

Question 4

[solution n°8 p.38]

Dans le fichier **main.py**, utilisez le module requests pour envoyer la requête et afficher les données reçues.

Indice :

Utilisez requests.get(url) pour envoyer une requête HTTP et response.json() pour afficher le contenu sous forme de dictionnaire.

Étape 5 : importation dynamique d'une configuration

José, souhaite désormais afficher les prix de ses produits en fonction de la devise du pays où se trouvent ses clients. En Europe, les prix doivent être affichés en euros (€), tandis qu'aux États-Unis, ils doivent être affichés en dollars ($). José vous demande de mettre en place un système d'importation dynamique qui lui permet de choisir la devise à afficher en fonction du marché.

1. https://jsonplaceholder.typicode.com/posts/1

Tous droits réservés à STUDI - Reproduction interdite

Question 5

[solution n°9 p.39]

Créez les fichiers de configuration et écrivez une fonction permettant d'importer dynamiquement la configuration de devise selon le marché (utilisez importlib).

Indice :

Utilisez importlib.import_module() pour importer dynamiquement le bon fichier de configuration en fonction de la devise sélectionnée.

B. Test

Exercice 1 : Quiz

[solution n°10 p.39]

Question 1

Quel est le mot-clé utilisé pour importer un module Python dans votre script ?

- ○ include
- ○ import
- ○ load
- ○ define

Question 2

Que se passe-t-il si un module est déjà importé et que vous tentez de l'importer à nouveau dans un même programme ?

- ○ Il est rechargé automatiquement
- ○ Une erreur est renvoyée
- ○ Il n'est pas rechargé, car il est mis en cache
- ○ Il est supprimé puis réimporté

Question 3

Quelle commande permet de générer automatiquement un fichier requirements.txt contenant toutes les dépendances d'un projet Python ?

- ○ pip generate
- ○ pip freeze
- ○ pip export
- ○ pip capture

Question 4

Quel est l'intérêt d'utiliser des modules standard comme math ou os au lieu d'écrire ses propres fonctions pour ces tâches ?

- ○ Ils sont plus rapides et optimisés
- ○ Ils permettent de supprimer les autres dépendances
- ○ Ils permettent de créer automatiquement des fichiers .py
- ○ Ils empêchent les erreurs dans le programme

Tous droits réservés à STUDI - Reproduction interdite

Question 5

Que permet le fichier __init__.py dans un package Python ?

- De créer un environnement virtuel
- D'initialiser un package et de le traiter comme un module
- De compiler les modules en bytecode
- De lancer un programme à l'exécution

Question 6

Quelle est la manière correcte de spécifier une version minimale d'un module dans un fichier requirements.txt ?

- module>=version
- module<=version
- module==version
- module!

Question 7

Quelle pratique est recommandée pour éviter les problèmes d'importation circulaire entre les modules Python ?

- Utiliser des noms de modules plus courts
- Limiter les dépendances entre modules et importer dans des fonctions si nécessaire
- Utiliser des fichiers .pyc compilés
- Utiliser des modules tiers à la place des modules internes

Question 8

Comment limiter le nombre de modules autorisés dans une importation dynamique avec importlib ?

- Utiliser un fichier importlib.conf
- Utiliser un environnement virtuel
- Utiliser une liste blanche des modules autorisés
- Déclarer les modules dans sys.modules

Question 9

Dans quel cas utiliser importlib.reload() ?

- Lors de la première importation d'un module
- Après avoir modifié le fichier source d'un module
- Lors de la suppression d'un module
- Lors du changement de version de Python

Question 10

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la meilleure pratique pour organiser un projet avec plusieurs modules en Python ?

- ○ Regrouper toutes les fonctions dans un seul fichier
- ○ Créer un package avec des sous-modules organisés par fonctionnalité
- ○ Mettre tous les modules dans un fichier compressé .zip
- ○ Utiliser des modules aléatoires à chaque exécution

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 4 Solution n°1

Question 1

Qu'est-ce qu'un module en Python ?

- ○ Une fonction prédéfinie du langage Python
- ◉ Un fichier contenant du code Python (fonctions, classes, variables) pouvant être importé et utilisé dans d'autres scripts
- ○ Une méthode pour optimiser la performance de Python
- ○ Un type de boucle spécifique à Python

Un module en Python est un fichier qui contient du code Python, comme des fonctions, des classes, ou des variables, qui peut être importé et utilisé dans d'autres fichiers Python pour favoriser la réutilisation du code.

Question 2

Pourquoi est-il recommandé d'utiliser des modules en Python ?

- ○ Pour éviter d'écrire du code
- ○ Pour simplifier l'installation de Python
- ◉ Pour organiser le code, favoriser la réutilisation et éviter les conflits de noms
- ○ Pour améliorer la vitesse de compilation

Les modules aident à organiser le code de manière logique, facilitent la réutilisation de code déjà écrit et aident à éviter les conflits de noms en créant des espaces de noms séparés.

Question 3

Quel mot-clé est utilisé pour importer un module en Python ?

- ○ include
- ○ using
- ◉ import
- ○ require

En Python, le mot-clé `import` est utilisé pour charger un module et rendre ses fonctions, classes ou variables accessibles dans le script actuel.

Question 4

Que se passe-t-il lorsque vous exécutez `import math` dans un script Python ?

- ○ Vous créez un nouveau module appelé `math`
- ○ Vous téléchargez le module `math` depuis Internet
- ◉ Vous importez le module `math` de la bibliothèque standard de Python
- ○ Vous supprimez toutes les variables mathématiques

Tous droits réservés à STUDI - Reproduction interdite

Le module `math` fait partie de la bibliothèque standard de Python. Lorsqu'on exécute `math`, on importe ce module et rend ses fonctions disponibles dans le script.

Question 5

Comment appelle-t-on un répertoire contenant plusieurs modules Python ainsi qu'un fichier `__init__.py` ?

- ○ Un projet
- ○ Un module principal
- ◉ Un package
- ○ Une fonction

Un package est un répertoire contenant plusieurs modules Python ainsi qu'un fichier `__init__.py`,, qui indique à Python que le répertoire doit être traité comme un package.

Question 6

Quelle est la fonction principale du fichier __init__.py dans un package Python ?

- ○ Empêcher le package d'être importé
- ◉ Permettre à Python de traiter le répertoire comme un package
- ○ Installer automatiquement tous les modules du package
- ○ Optimiser les performances du package

Le fichier __init__.py permet à Python de reconnaître un répertoire comme un package et d'exécuter tout code d'initialisation nécessaire à l'importation de ce package.

Question 7

Quelle commande utilisez-vous pour installer un module tiers en Python ?

- ○ python install
- ◉ pip install
- ○ module install
- ○ module install

La commande pip install` est utilisée pour installer des modules tiers depuis le Python Package Index (PyPI).

Question 8

Quel est le résultat de l'exécution du code suivant ?

```
1 import calcul
2 print(calcul.addition(5, 7))
```

Considérez que le module `calcul` contient une fonction `addition(a, b)` qui retourne la somme de `a` et `b`.

- ○ Le programme ne s'exécutera pas car le module `calcul` n'est pas défini
- ○ Le programme affiche : `57`
- ◉ Le programme affiche : `12`
- ○ Le programme génère une erreur de syntaxe

Tous droits réservés à STUDI - Reproduction interdite

Si le module `calcul` est correctement défini et contient la fonction `addition`, le programme affichera `12` car la fonction additionne 5 et 7.

Question 9

Pourquoi est-il utile d'utiliser le module `requests` en Python ?

- ○ Pour manipuler les fichiers locaux sur l'ordinateur
- ◉ Pour effectuer des requêtes HTTP, comme récupérer des données d'un site web
- ○ Pour gérer les expressions régulières dans les chaînes de caractères
- ○ Pour améliorer la performance de l'application Python

Le module `requests` est utilisé pour effectuer des requêtes HTTP facilement, permettant aux développeurs de communiquer avec des serveurs web et de récupérer ou envoyer des données.

Question 10

Quelle est la différence principale entre l'utilisation de import module et from module import * ?

- ○ import module importe le module entier, tandis que from module import * importe uniquement le nom du module
- ◉ import module importe le module entier, tandis que from module import * importe toutes les fonctions et classes du module dans l'espace de noms actuel
- ○ import module est obsolète et ne devrait plus être utilisé
- ○ Les deux sont exactement identiques et peuvent être utilisés de manière interchangeable

import module importe le module entier et nécessite l'utilisation de la syntaxe module.nom_fonction, tandis que from module import * importe toutes les fonctions et classes du module dans l'espace de noms actuel, ce qui peut potentiellement causer des conflits de noms.

Exercice p. 11 Solution n°2

Question 1

Quel est le rôle principal du fichier __init__.py dans un package Python ?

- ○ Il doit contenir toutes les fonctions du package
- ○ Il empêche le package d'être importé
- ◉ Il indique à Python que le répertoire est un package
- ○ Il permet de configurer les variables globales du projet

Le fichier __init__.py indique à Python que le répertoire dans lequel il se trouve doit être traité comme un package. Il peut être vide ou contenir du code d'initialisation.

Question 2

Que permet l'instruction from math_operations import * ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Elle importe seulement les fonctions spécifiées de math_operations
- ◉ Elle importe toutes les fonctions et classes de math_operations sans utiliser le préfixe math_operations
- ○ Elle importe uniquement les variables globales définies dans math_operations
- ○ Elle crée une copie du module math_operations

Q L'instruction from ... import * importe tout le contenu du module sans préfixe, ce qui peut causer des conflits de noms si plusieurs modules ont des fonctions similaires.

Question 3

Quelle méthode est recommandée pour importer des modules dans des projets complexes ?

- ○ Utiliser from module import * pour chaque module
- ◉ Utiliser import module_name ou importer des fonctions spécifiques avec from module import fonction
- ○ Tout inclure dans un seul fichier pour éviter les imports
- ○ Importer des modules à la fin du script

Q Il est recommandé d'utiliser import module_name pour garder le code clair et éviter les conflits de noms, ou bien d'importer des fonctions spécifiques explicitement.

Question 4

Pourquoi est-il déconseillé d'utiliser from ... import * dans un projet avec plusieurs modules ?

- ○ Cela réduit les performances du projet
- ◉ Cela peut entraîner des conflits de noms si plusieurs modules ont des fonctions identiques
- ○ Cela rend impossible l'importation de classes
- ○ Cela empêche Python de trouver les modules dans la bibliothèque standard

Q from ... import * importe tout le contenu d'un module dans l'espace de noms global, ce qui peut entraîner des conflits si plusieurs modules ont des fonctions ou des variables portant le même nom.

Question 5

Quelle est la différence entre un module et un package en Python ?

- ○ Un module est une collection de packages
- ◉ Un package est un répertoire contenant plusieurs modules et un fichier __init__.py
- ○ Un package est un fichier Python unique
- ○ Un module ne peut contenir que des classes, alors qu'un package contient des fonctions

Q Un package est un répertoire contenant plusieurs modules ainsi qu'un fichier spécial __init__.py, tandis qu'un module est un fichier Python individuel.

Question 6

Comment appelle-t-on l'importation basée sur la position relative des modules ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Importation différée
- ○ Importation absolue
- ◉ Importation relative
- ○ Importation cyclique

L'importation relative utilise des points pour indiquer la position d'un module par rapport au module courant dans la hiérarchie des dossiers.

Question 7

Comment créer un module Python ?

- ◉ En écrivant du code dans un fichier avec l'extension .py
- ○ En créant un répertoire contenant un fichier __init__.py
- ○ En exécutant pip install module_name
- ○ En utilisant la commande create module dans Python

Un module est simplement un fichier contenant du code Python avec l'extension .py. Il peut ensuite être importé dans d'autres scripts Python.

Question 8

Quelle est la principale raison de structurer un projet en modules et packages ?

- ○ Accélérer le développement
- ○ Réduire la taille du fichier .pyc
- ◉ Organiser le code pour le rendre plus maintenable, réutilisable et lisible
- ○ Obtenir un meilleur support de la part de la communauté Python

L'organisation du code en modules et packages permet de rendre le projet plus lisible, maintenable, réutilisable et structuré.

Question 9

Que permet l'instruction import module_name ?

- ○ Elle exécute le fichier spécifié dans le module
- ◉ Elle charge le module dans le script actuel, permettant d'utiliser ses fonctions et classes
- ○ Elle renomme le module
- ○ Elle importe uniquement les classes du module

L'instruction import module_name charge le module dans le script courant, rendant accessibles ses fonctions, classes et variables.

Question 10

Qu'est-ce qu'un conflit de noms en Python ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Lorsque deux modules portent le même nom
- ◉ Lorsque deux fonctions ou variables avec le même nom existent dans le même espace de noms
- ○ Lorsque un fichier Python a le même nom qu'un fichier de configuration
- ○ Lorsque deux packages sont importés en même temps

Q Un conflit de noms se produit lorsque deux éléments (fonctions, variables, classes) avec le même nom coexistent dans le même espace de noms, ce qui peut entraîner des erreurs ou des comportements inattendus.

Exercice p. 16 **Solution n°3**

Question 1

Quel module Python permet de gérer les fichiers et répertoires du système ?

- ○ sys
- ◉ os
- ○ file
- ○ path

Q Le module `os` permet d'interagir avec le système de fichiers pour gérer les répertoires et les fichiers.

Question 2

Quelle commande permet d'installer un module tiers depuis le Python Package Index (PyPI) ?

- ○ python install
- ◉ pip install
- ○ py install
- ○ pip freeze

Q La commande `pip install nom_du_package` installe un module tiers depuis PyPI.

Question 3

Que contient un fichier `requirements.txt` ?

- ○ La documentation du projet
- ○ Les fichiers exécutables du projet
- ◉ La liste des dépendances (modules) du projet avec leurs versions
- ○ Les paramètres de configuration du projet

Q Un fichier `requirements.txt` contient la liste des modules et dépendances du projet avec leurs versions spécifiques, permettant à d'autres développeurs de recréer le même environnement.

Question 4

Quelle est la commande utilisée pour installer les modules spécifiés dans un fichier `requirements.txt` ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ pip install requirements.txt
- ○ pip download -r requirements.txt
- ◉ pip install -r requirements.txt
- ○ python install -r requirements.txt

La commande `pip install -r requirements.txt` installe toutes les dépendances listées dans le fichier `requirements.txt` avec leurs versions exactes.

Question 5

Quel est l'avantage principal de figer les versions des modules dans un projet ?

- ○ Cela permet d'accélérer l'installation des modules
- ◉ Cela permet de s'assurer que le projet est compatible avec les versions testées des modules
- ○ Cela réduit la taille des fichiers du projet
- ○ Cela désactive les mises à jour automatiques

Figer les versions dans un fichier comme `requirements.txt` garantit que toutes les personnes travaillant sur le projet utilisent les mêmes versions testées des modules, assurant ainsi la stabilité et la compatibilité du projet.

Question 6

Que fait la commande `pip freeze` ?

- ○ Elle supprime tous les modules installés dans l'environnement Python
- ○ Elle met à jour tous les modules installés à leur dernière version
- ◉ Elle affiche la liste des modules installés avec leurs versions
- ○ Elle bloque les modules pour empêcher leur mise à jour

La commande `pip freeze` génère une liste de tous les modules installés dans l'environnement actuel avec leurs versions, souvent utilisée pour créer un fichier `requirements.txt`.

Question 7

Quelle commande permet de voir quelles versions de packages sont obsolètes dans l'environnement Python ?

- ◉ pip list --outdated
- ○ pip upgrade --list
- ○ pip freeze --upgrade
- ○ pip list --old

La commande pip list --outdated affiche les modules installés dans l'environnement actuel et qui peuvent être mis à jour.

Question 8

Pourquoi est-il important d'utiliser un fichier requirements.txt dans un projet Python ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Pour réduire la taille du projet
- ○ Pour améliorer la sécurité
- ◉ Pour garantir que tous les développeurs utilisent les mêmes versions de packages
- ○ Pour accélérer les performances du projet

Utiliser un fichier requirements.txt permet de garantir que tous les membres du projet installent les mêmes versions de packages, assurant ainsi la cohérence et la compatibilité du projet.

Question 9

Pourquoi est-il recommandé d'utiliser un environnement virtuel pour installer des modules dans un projet Python ?

- ○ Pour améliorer la vitesse d'exécution du code
- ◉ Pour isoler les dépendances d'un projet et éviter les conflits entre différents projets
- ○ Pour faciliter l'installation des modules
- ○ Pour réduire la taille des fichiers du projet

L'utilisation d'un environnement virtuel permet d'isoler les dépendances d'un projet, évitant ainsi les conflits entre différentes versions de modules utilisés dans différents projets sur la même machine.

Question 10

Quelle est la différence entre pip install et pip install -r requirements.txt ?

- ◉ pip install installe un module, tandis que pip install -r requirements.txt installe plusieurs modules à partir du fichier requirements.txt
- ○ pip install désinstalle un module, tandis que pip install -r requirements.txt installe un module
- ○ pip install met à jour tous les modules, tandis que pip install -r requirements.txt les désinstalle
- ○ pip install crée un fichier requirements.txt, tandis que pip install -r requirements.txt ne fait rien

pip install nom_du_module installe un module spécifique, tandis que pip install -r requirements.txt installe toutes les dépendances listées dans un fichier requirements.txt.

Exercice p. 20 Solution n°4

Question 1

Quel est l'avantage principal de l'importation dynamique avec `importlib.import_module()` ?

- ○ Elle permet de réduire la taille des fichiers du projet
- ◉ Elle permet de charger des modules uniquement au moment de leur utilisation
- ○ Elle permet de désinstaller des modules à l'exécution
- ○ Elle empêche l'importation de modules non sécurisés

L'importation dynamique permet de retarder le chargement des modules jusqu'au moment où ils sont nécessaires, ce qui améliore la modularité et la flexibilité du programme.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Quelle fonction du module `importlib` permet de recharger un module déjà importé ?

- ○ importlib.import_module()
- ○ importlib.load_module()
- ◉ importlib.reload()
- ○ reload.module()

`importlib.reload()` permet de recharger un module déjà importé, utile dans les environnements de développement.

Question 3

Quel est le risque principal de l'importation dynamique non sécurisée de modules ?

- ○ Provoquer des erreurs d'exécution
- ○ Importer des modules inutiles
- ◉ Exécuter du code malveillant présent dans un module non approuvé
- ○ Réduire les performances globales du programme

L'importation dynamique de modules non approuvés peut conduire à l'exécution de code malveillant inclus dans le module.

Question 4

Quel module Python est utilisé pour l'importation dynamique de modules ?

- ○ os
- ◉ importlib
- ○ math
- ○ random

Le module `importlib` est utilisé pour importer des modules de manière dynamique au moment de l'exécution.

Question 5

Quel est le rôle principal de la fonction `importlib.import_module()` ?

- ○ Recharger un module modifié
- ◉ Importer un module en utilisant une chaîne de caractères
- ○ Supprimer un module non utilisé
- ○ Lister les modules disponibles dans l'environnement

`importlib.import_module()` permet d'importer un module à l'exécution en utilisant son nom sous forme de chaîne de caractères.

Question 6

Que se passe-t-il si un module est déjà importé et que vous essayez de le recharger avec `importlib.reload()` ?

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ Le module est rechargé avec les nouvelles modifications
- ○ Le module est désinstallé puis réinstallé
- ○ Le module est rechargé mais ses objets existants ne sont pas mis à jour
- ○ Le programme plante automatiquement

Q `importlib.reload()` permet de recharger un module déjà importé avec les nouvelles modifications.

Question 7

Dans quel cas l'importation dynamique de modules est-elle particulièrement utile ?

- ○ Lorsque le module doit être modifié après l'importation
- ⦿ Lorsque le module à importer dépend d'une configuration ou d'un contexte spécifique
- ○ Lorsque vous voulez accélérer l'importation d'un module
- ○ Lorsque vous importez des modules depuis Internet

Q L'importation dynamique est particulièrement utile lorsque le choix du module à importer dépend d'une configuration spécifique ou d'une condition qui n'est connue qu'à l'exécution.

Question 8

Quelle est la meilleure pratique pour sécuriser l'importation dynamique de modules ?

- ○ Importer tous les modules disponibles dans l'environnement
- ⦿ Utiliser une liste blanche de modules autorisés
- ○ Utiliser uniquement `import` au lieu de `importlib`
- ○ Importer des modules directement depuis Internet

Q Utiliser une liste blanche pour restreindre les modules qui peuvent être importés est une bonne pratique pour éviter l'importation de modules malveillants.

Question 9

Pourquoi est-il déconseillé d'importer des modules provenant de sources non fiables ?

- ○ Cela peut ralentir l'exécution du programme
- ⦿ Le code contenu dans ces modules peut être malveillant et compromette la sécurité
- ○ Cela provoque des erreurs d'importation
- ○ Cela surcharge la mémoire du programme

Q Importer des modules depuis des sources non fiables peut exécuter du code malveillant qui pourrait compromettre la sécurité du programme.

Question 10

Comment restreindre les modules qui peuvent être importés dynamiquement ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ En limitant l'importation aux modules du système
- ◉ En définissant une liste blanche de modules approuvés
- ○ En désactivant les imports dans le projet
- ○ En évitant l'utilisation de `importlib`

Une liste blanche permet de restreindre l'importation aux seuls modules autorisés, réduisant ainsi les risques de chargement de modules malveillants.

p. 23 Solution n°5

Dans le fichier **commandes.py**, vous devez créer deux fonctions : ajouter_commande() et afficher_commandes(). La liste commandes est globale pour stocker toutes les commandes.

```
1 # Liste globale des commandes
2 commandes = []
3
4 def ajouter_commande(client, produit, prix):
5     """Ajoute une commande à la liste des commandes."""
6     commande = {
7         'client': client,
8         'produit': produit,
9         'prix': prix
10     }
11     commandes.append(commande)
12
13 def afficher_commandes():
14     """Affiche toutes les commandes."""
15     for commande in commandes:
16         print(f"Client: {commande['client']}, Produit: {commande['produit']}, Prix:
{commande['prix']}")
```

p. 23 Solution n°6

Dans le fichier **main.py**, vous importez le module commandes et utilisez les fonctions ajouter_commande() et afficher_commandes() pour ajouter et afficher des commandes.

```
1 import commandes
2
3 # Ajouter des commandes
4 commandes.ajouter_commande("Alice", "Laptop", 1200)
5 commandes.ajouter_commande("Bob", "Smartphone", 800)
6 commandes.ajouter_commande("Charlie", "Tablette", 300)
7
8 # Afficher les commandes
9 commandes.afficher_commandes()
```

La sortie du code, donnera cela :

```
1 Client: Alice, Produit: Laptop, Prix: 1200
2 Client: Bob, Produit: Smartphone, Prix: 800
3 Client: Charlie, Produit: Tablette, Prix: 300
```

p. 23 Solution n°7

Tous droits réservés à STUDI - Reproduction interdite

```python
import datetime  # Import du module datetime

commandes = []

def ajouter_commande(client, produit, prix):
    """Ajoute une commande avec la date à la liste des commandes."""
    date_commande = datetime.datetime.now()  # Obtenir la date actuelle
    commande = {
        'client': client,
        'produit': produit,
        'prix': prix,
        'date': date_commande
    }
    commandes.append(commande)

def afficher_commandes():
    """Affiche toutes les commandes."""
    for commande in commandes:
        print(f"Client: {commande.get('client')}, Produit: {commande.get('produit')}, Prix: {commande.get('prix')}, Date: {commande.get('date')}")
```

```
Client: Alice, Produit: Laptop, Prix: 1200, Date: 2024-09-22 14:56:12.123456
Client: Bob, Produit: Smartphone, Prix: 800, Date: 2024-09-22 14:56:15.789012
Client: Charlie, Produit: Tablette, Prix: 300, Date: 2024-09-22 14:56:18.345678
```

Nous avons modifié la fonction **ajouter_commande()** pour inclure la date actuelle de la commande en utilisant le module **datetime**.

Explication de la solution :

1. **Importation du module datetime** :

   Le module datetime fait partie des modules standards de Python. Il fournit des fonctions pour manipuler les dates et les heures. Nous avons importé ce module en haut du fichier **commandes.py** pour l'utiliser dans la fonction ajouter_commande().

2. **Ajout de la date à la commande** :

   Lors de l'ajout d'une commande, nous avons utilisé la méthode **datetime.datetime.now()** pour obtenir la date et l'heure actuelles. Cette valeur est stockée dans une clé 'date' du dictionnaire représentant la commande.

3. **Affichage de la date dans afficher_commandes()** :

   La fonction d'affichage a été modifiée pour inclure la date de chaque commande dans le format de sortie.

p. 23 **Solution n°8**

```python
import requests

# Envoyer une requête GET
url = 'https://jsonplaceholder.typicode.com/posts/1'
response = requests.get(url)

# Vérifier si la requête a réussi (statut 200)
if response.status_code == 200:
    # Afficher le contenu JSON de la réponse
    print(response.json())
else:
    print(f"Erreur : {response.status_code}")
```


Tous droits réservés à STUDI - Reproduction interdite

p. 24 Solution n°9

Créez deux fichiers de configuration : **config_euro.py** et **config_usd.py**, contenant respectivement une variable DEVISE de valeur « € » ou « $ ».

Puis, utilisons le dans le main.py

```
import importlib
import commandes

def charger_devise(pays):
    """Charge dynamiquement le module de configuration pour la devise."""
    module_name = f'config_{pays}'
    try:
        config = importlib.import_module(module_name)
        return config
    except ImportError:
        print(f"Erreur : Le module pour le pays '{pays}' n'a pas pu être chargé.")
        return None

# Exemple d'utilisation :
pays = 'euro'  # Ou 'usd' selon le marché
config = charger_devise(pays)

# Ajouter des commandes et afficher les prix avec la bonne devise
if config:
    commandes.ajouter_commande("Alice", "Laptop", 1200)
    commandes.ajouter_commande("Bob", "Smartphone", 800)

    # Afficher les commandes avec la devise
    for commande in commandes.commandes:
        print(f"Client: {commande.get('client')}, Produit: {commande.get('produit')}, Prix: {commande.get('prix')}{config.DEVISE}")
```

Explication de la solution :

1. **Création des fichiers de configuration :** nous avons créé deux fichiers, **config_euro.py** et **config_usd.py**, contenant chacun une variable **DEVISE** qui indique la devise à utiliser.
2. **Importation dynamique :** la fonction charger_devise() utilise importlib.import_module() pour importer dynamiquement le module de configuration en fonction du paramètre pays passé. Ce paramètre détermine si la devise choisie sera l'euro ou le dollar.
3. **Utilisation de la devise :** après avoir importé la configuration, les prix des produits sont affichés avec la devise correspondante.

Exercice p. 24 Solution n°10

Question 1

Quel est le mot-clé utilisé pour importer un module Python dans votre script ?

- ○ include
- ◉ import
- ○ load
- ○ define

Tous droits réservés à STUDI - Reproduction interdite

Le mot-clé import permet d'importer un module en Python.

Question 2

Que se passe-t-il si un module est déjà importé et que vous tentez de l'importer à nouveau dans un même programme ?

- ○ Il est rechargé automatiquement
- ○ Une erreur est renvoyée
- ◉ Il n'est pas rechargé, car il est mis en cache
- ○ Il est supprimé puis réimporté

Python utilise un mécanisme de cache pour éviter de réimporter plusieurs fois un module déjà chargé.

Question 3

Quelle commande permet de générer automatiquement un fichier requirements.txt contenant toutes les dépendances d'un projet Python ?

- ○ pip generate
- ◉ pip freeze
- ○ pip export
- ○ pip capture

pip freeze > requirements.txt génère un fichier listant les dépendances d'un projet avec leurs versions.

Question 4

Quel est l'intérêt d'utiliser des modules standard comme math ou os au lieu d'écrire ses propres fonctions pour ces tâches ?

- ◉ Ils sont plus rapides et optimisés
- ○ Ils permettent de supprimer les autres dépendances
- ○ Ils permettent de créer automatiquement des fichiers .py
- ○ Ils empêchent les erreurs dans le programme

Les modules standards sont optimisés et testés, ce qui les rend souvent plus efficaces que des implémentations manuelles.

Question 5

Que permet le fichier __init__.py dans un package Python ?

- ○ De créer un environnement virtuel
- ◉ D'initialiser un package et de le traiter comme un module
- ○ De compiler les modules en bytecode
- ○ De lancer un programme à l'exécution

Tous droits réservés à STUDI - Reproduction interdite

Le fichier __init__.py indique à Python qu'un dossier doit être traité comme un package et permet d'initialiser ce package.

Question 6

Quelle est la manière correcte de spécifier une version minimale d'un module dans un fichier requirements.txt ?

- (●) module>=version
- ( ) module<=version
- ( ) module==version
- ( ) module!

Utiliser module>=version dans le fichier requirements.txt garantit que la version installée sera au moins celle spécifiée, mais pourra être une version plus récente.

Question 7

Quelle pratique est recommandée pour éviter les problèmes d'importation circulaire entre les modules Python ?

- ( ) Utiliser des noms de modules plus courts
- (●) Limiter les dépendances entre modules et importer dans des fonctions si nécessaire
- ( ) Utiliser des fichiers .pyc compilés
- ( ) Utiliser des modules tiers à la place des modules internes

Pour éviter les imports circulaires, il est préférable de limiter les dépendances entre modules ou de retarder l'importation en l'intégrant dans une fonction.

Question 8

Comment limiter le nombre de modules autorisés dans une importation dynamique avec importlib ?

- ( ) Utiliser un fichier importlib.conf
- ( ) Utiliser un environnement virtuel
- (●) Utiliser une liste blanche des modules autorisés
- ( ) Déclarer les modules dans sys.modules

Une liste blanche des modules autorisés permet de restreindre les importations dynamiques aux seuls modules validés.

Question 9

Dans quel cas utiliser importlib.reload() ?

- ( ) Lors de la première importation d'un module
- (●) Après avoir modifié le fichier source d'un module
- ( ) Lors de la suppression d'un module
- ( ) Lors du changement de version de Python

importlib.reload() est utilisé pour recharger un module après l'avoir modifié, sans redémarrer l'interpréteur Python.

Tous droits réservés à STUDI - Reproduction interdite

Question 10

Quelle est la meilleure pratique pour organiser un projet avec plusieurs modules en Python ?

- ○ Regrouper toutes les fonctions dans un seul fichier
- ◉ Créer un package avec des sous-modules organisés par fonctionnalité
- ○ Mettre tous les modules dans un fichier compressé .zip
- ○ Utiliser des modules aléatoires à chaque exécution

Organiser le projet en packages et sous-modules par fonctionnalité est une bonne pratique pour maintenir la lisibilité et la maintenance du code.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite